Description of the models and Implementation :

Direct coordinate regression :

I used a simple CNN with 4 convolutional layers, batch normalization and dropout and achieved moderately good results. The model directly regresses 68 facial keypoint (x, y) coordinates from a grayscale 224×224 input image.

Transfer Learning for Keypoint Detection :

For the transfer learning part I used nn.Identity to remove the models classification head and added a simple regression head with the output being a vector of (batch, keypoint_size * 2) shape.

I also switched the first conv layer from 3→1 input channels (grayscale) by repeating the first channel weights across all 3 channels.

Architecture :

- Regression Head: Three FC layers: Linear(512 → 1024) → ReLU → Dropout(0.5) → Linear(1024 → 512) → ReLU → Dropout(0.5) → Linear(512 → 136). Outputs 68×2 coordinates.

I created the _freeze_backbone and _unfreeze_backbone methods to quickly switch the parameters of the model backbone from require_grad. This allows for easier training of the two stages, where the only difference is how the optimizer is set up.

Training strategy (two-stage):

- Stage 1 (frozen): Backbone parameters are frozen; only the regression head is trained (lr=5e-4). This lets the head learn to map pretrained features to keypoints.

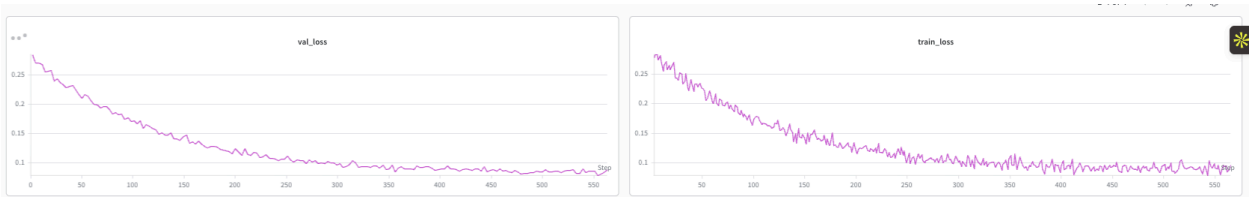
- Stage 2 (fine-tuning): All layers are unfrozen and trained end-to-end with a lower lr (1e-4) to adapt the backbone features to the keypoint task.

DINO :

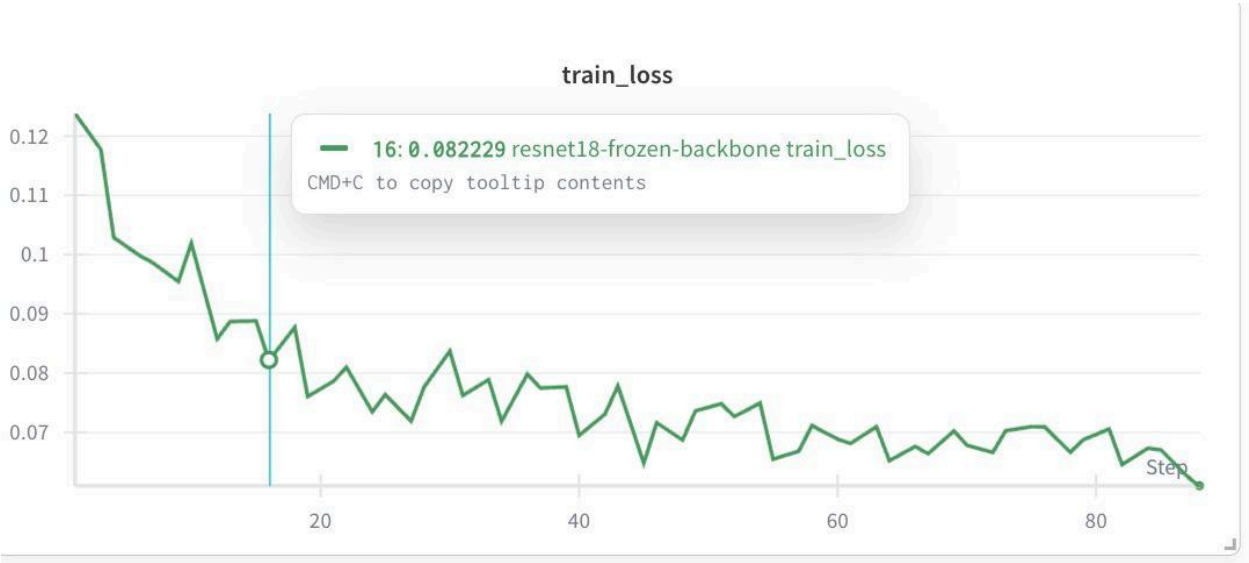
Very similar implementation to ResNet with the main difference being : DINO was pretrained with self-supervised contrastive learning (no labels), producing features that capture richer structural/spatial information compared to supervised ImageNet classification features. This can in theory be advantageous for keypoint localization since DINO features tend to encode part-level semantics. But in practice I found the ResNet implementation to be more accurate.

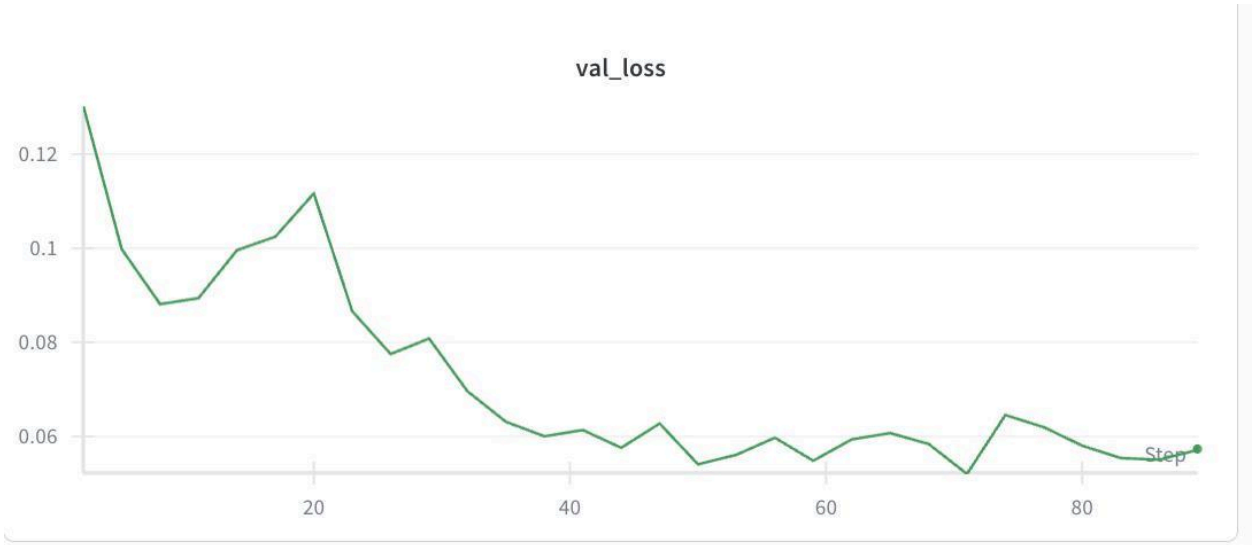
Training and validation loss curves :

CNN :

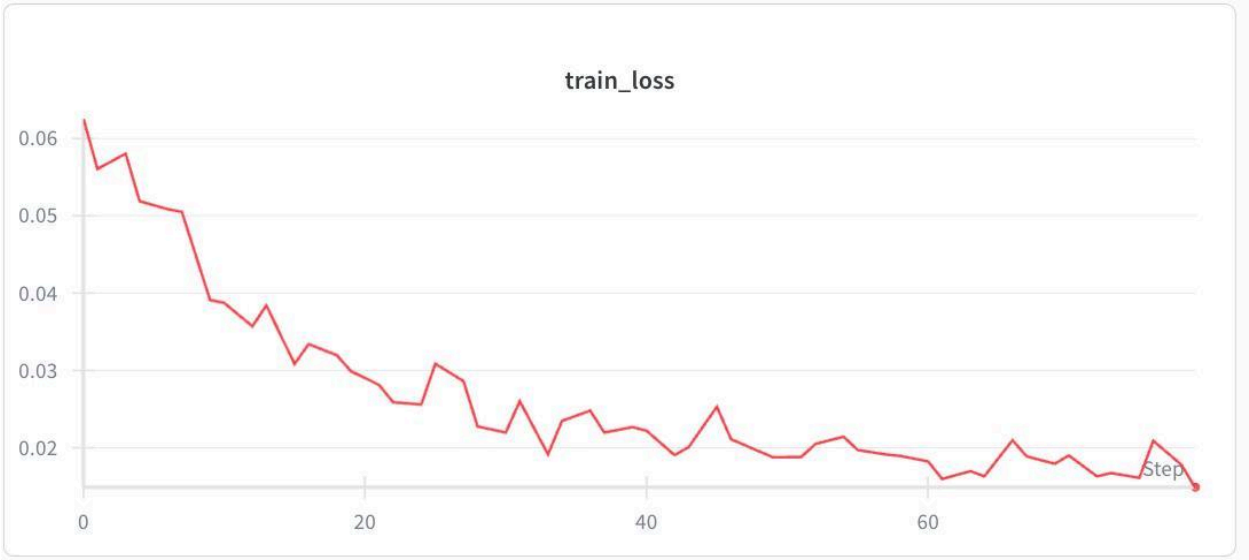


Resnet - freezed pretrained model (phase 1) :



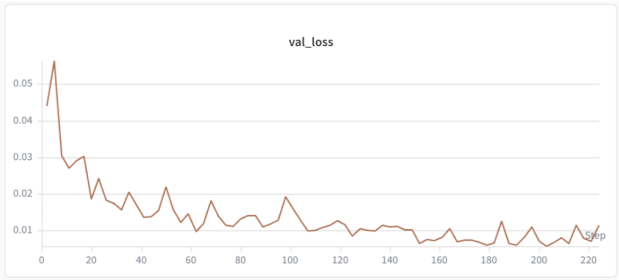


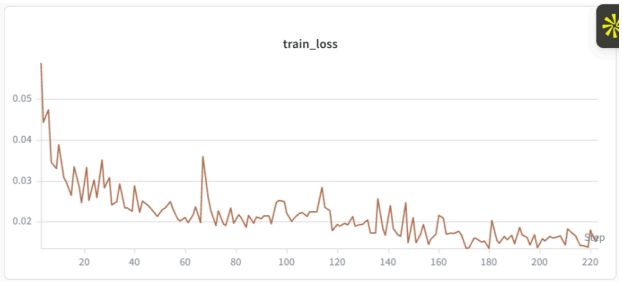
Resnet - Full Finetuning :



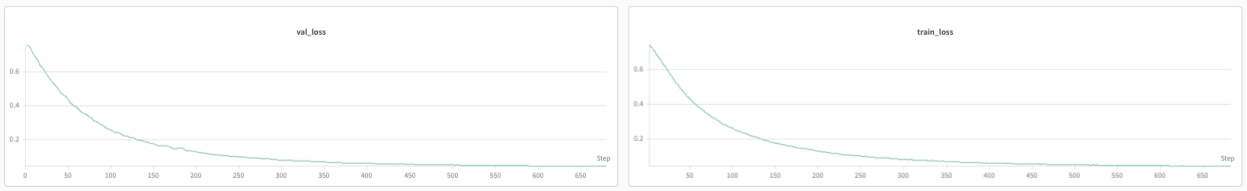
DINO - Freezed backbone :

DINO - Full Finetuning



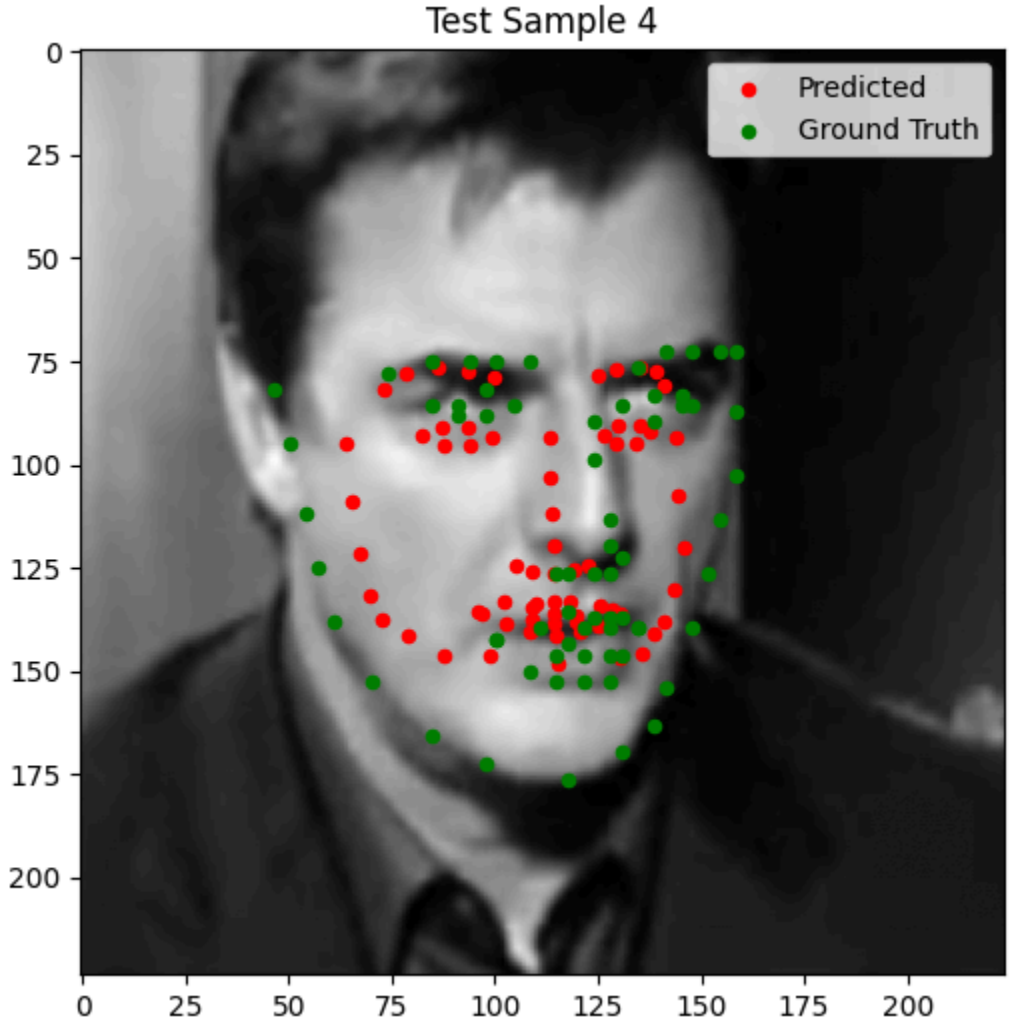


Unet :

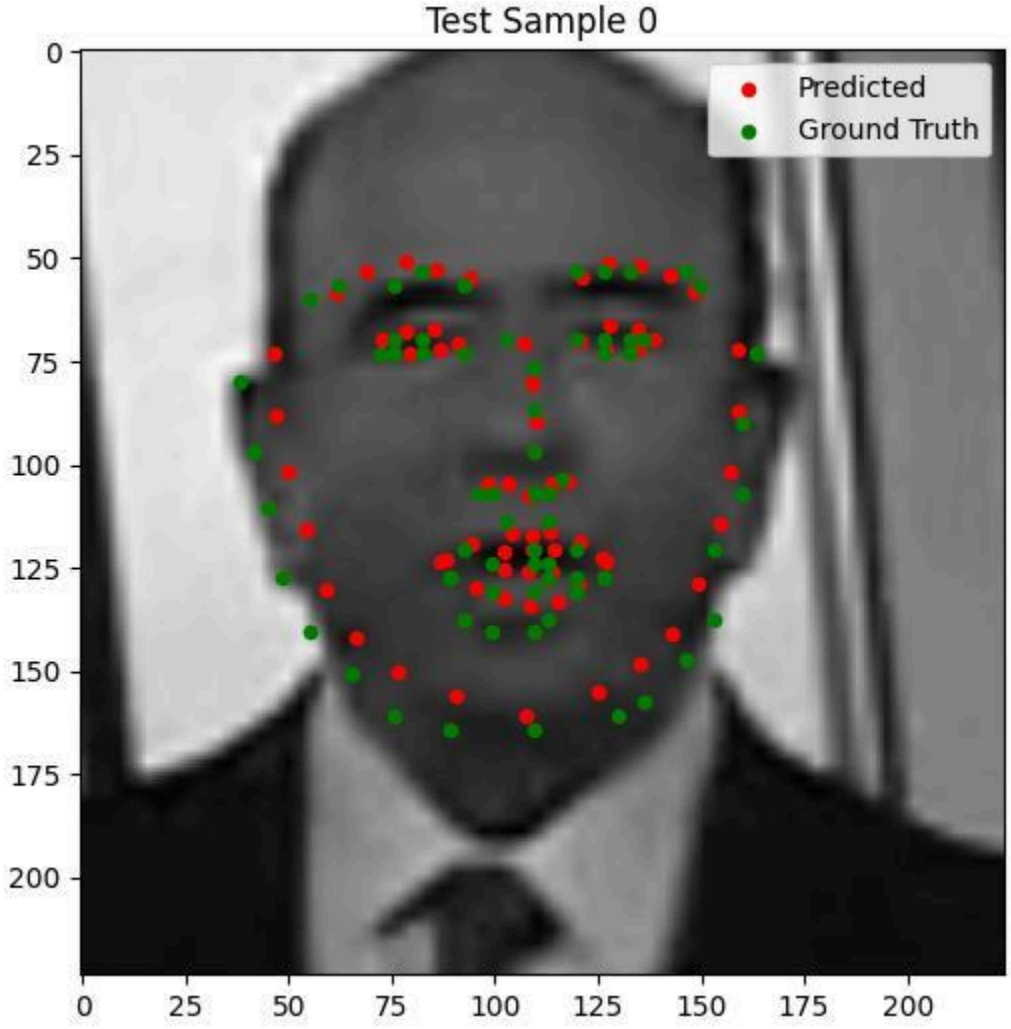


Visualization of predictions on test images

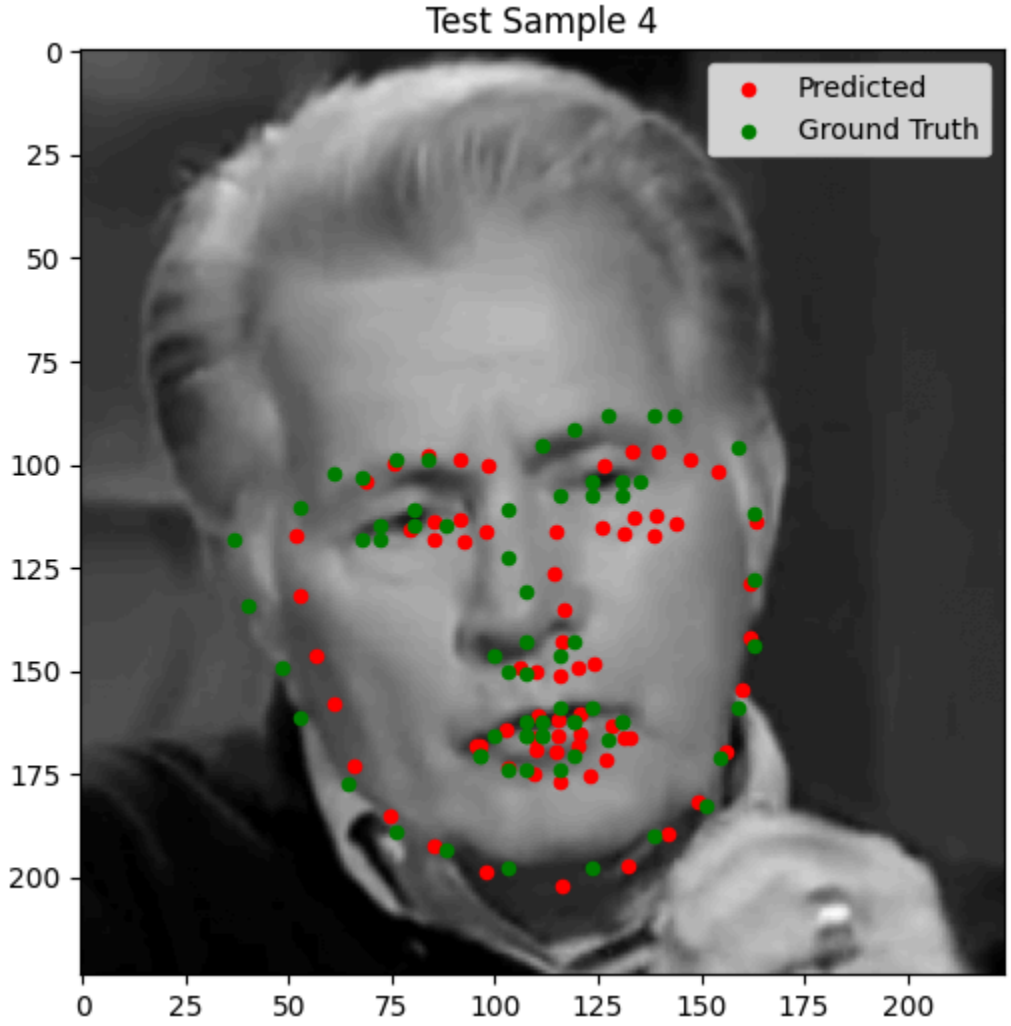
CNN :


Test Sample 4
Predicted
Ground Truth
0
25
50
75
100
125
150
175
200

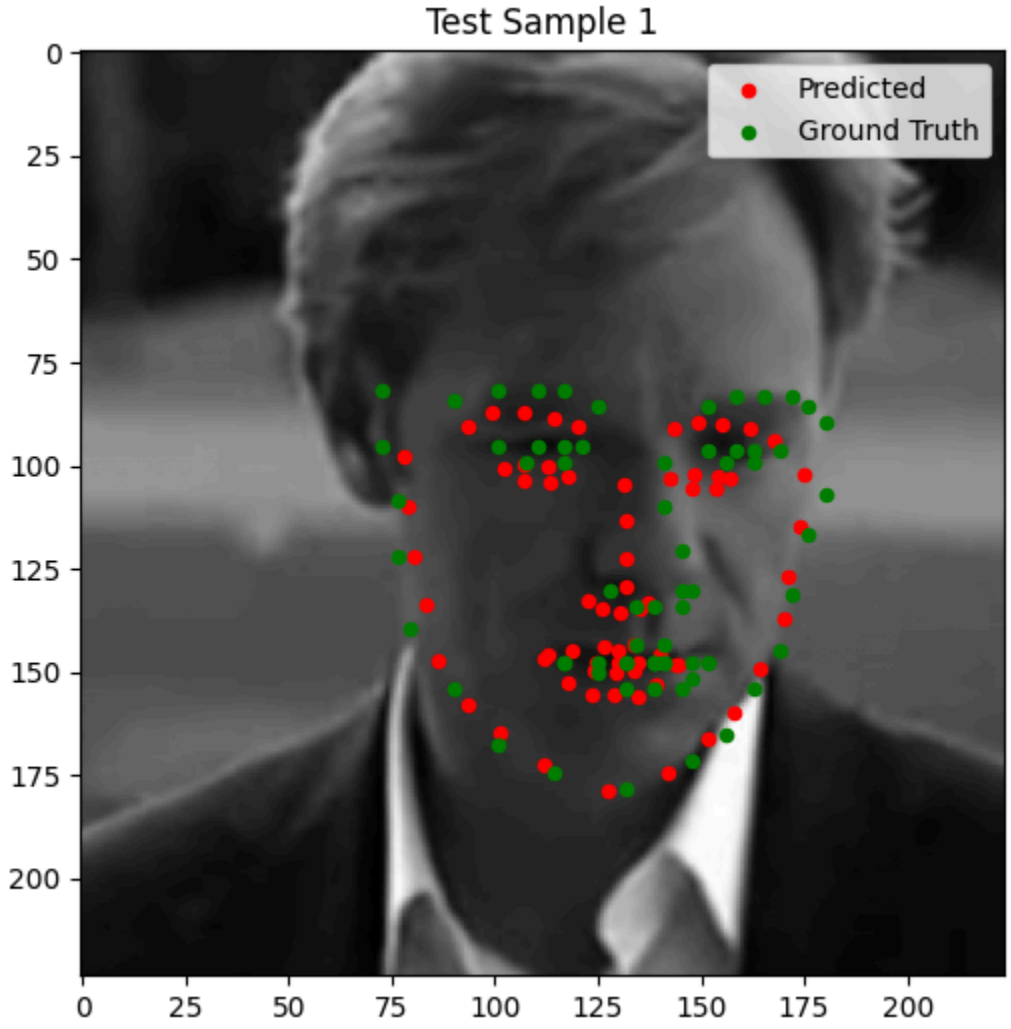
ResNet :

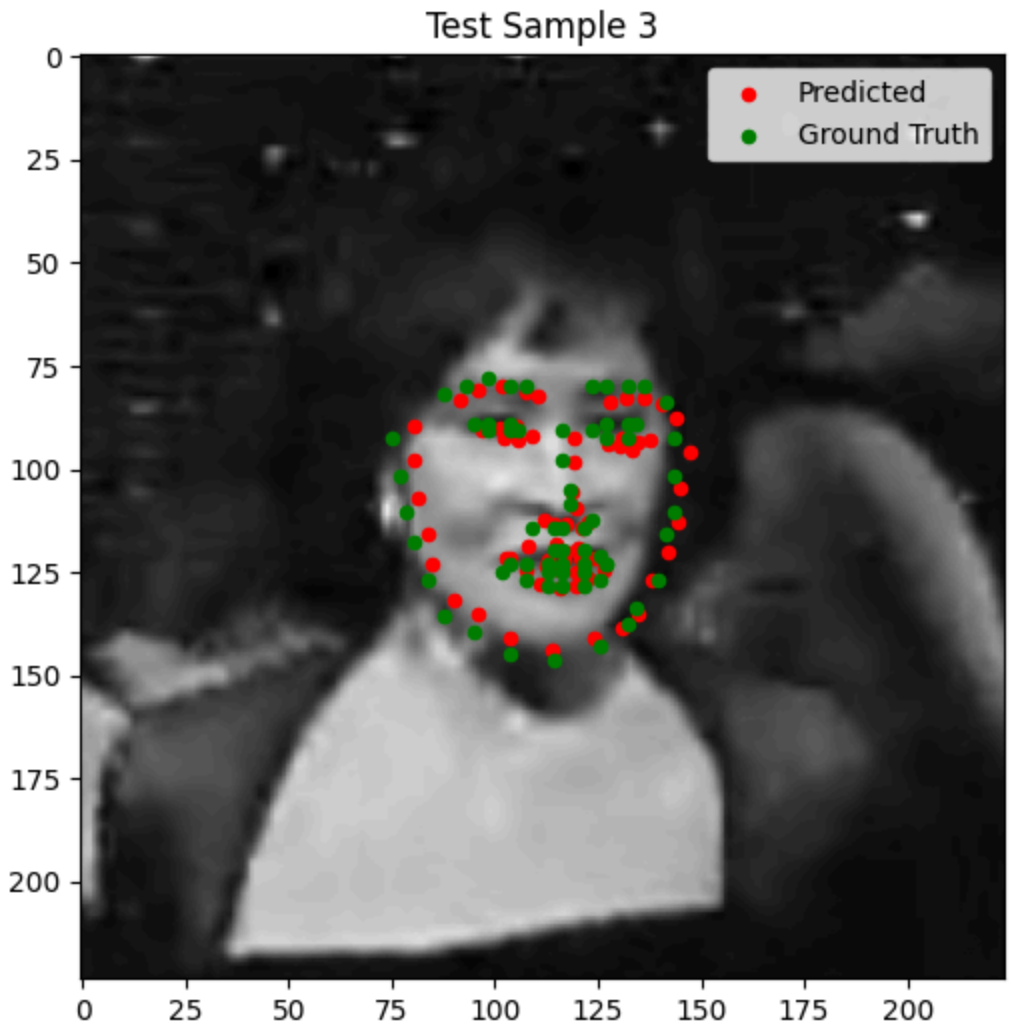

Test Sample 0
Predicted
Ground Truth
0
25
50
75
100
125
150
175
200

DINO - Freezed :

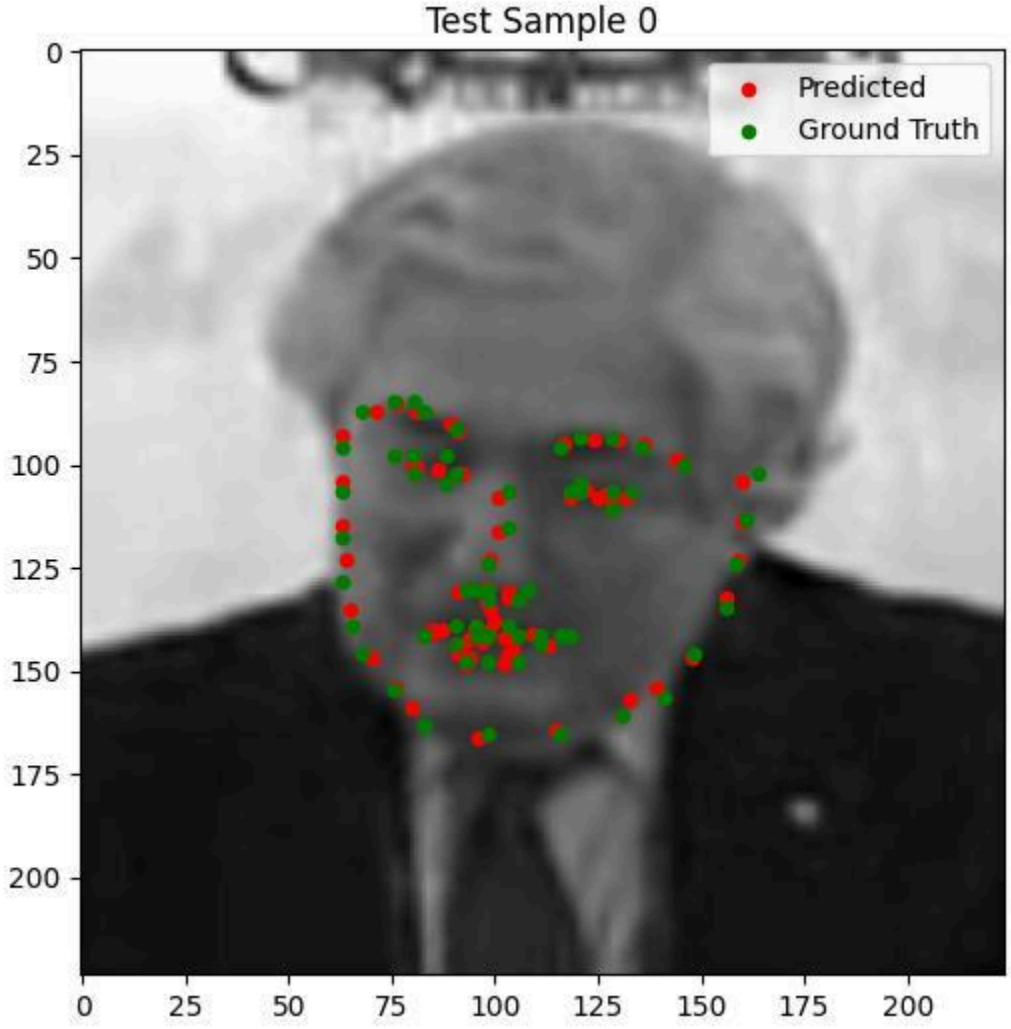


DINO - Full Finetuning :




Test Sample 3
Predicted
Ground Truth
0
25
50
75
100
125
150
175
200

Unet :


Test Sample 0
Predicted
Ground Truth
0
25
50
75
100
125
150
175
200

Strengths and weaknesses of each method :

1) Direct regression (CNN) :

Strengths

- easy to implement and debug
- **Fast training and inference** due to small output space (only 136 values)
- **Low memory usage** compared to heatmap models
- **Works reasonably well on clean, centered faces**

Weaknesses

- **Ignores spatial structure** of keypoints (outputs coordinates independently)
- **Hard for model to learn precise localization** since it must map entire image → coordinates directly
- **Sensitive to scale/translation changes** unless heavily augmented
- **Low accuracy**

2) Transfer Learning (Pretrained Backbone)

Strengths

- **Leverages rich visual features learned from large datasets** (edges, textures, shapes, semantics)
- **Much faster convergence** than training from scratch. Loss curve started at a lower value compared to CNN
- **Better generalization** on small datasets
- **Usually significantly more accurate than a simple CNN**

Weaknesses

- **Heavier model requires more GPU memory + slower training**
- **Feature mismatch**: pretrained networks are trained for classification, not localization But the Stage1 training with frozen backbone speeds up the training

3) U-net Heatmap prediction :

Strengths

- **Explicit spatial modeling**: network predicts the heatmap which is easier than predicting the keypoints directly
- **More robust to pose changes, occlusions, and scale variations**
- **Skip connections preserve fine spatial detail**
- **Closer to modern state-of-the-art pose estimation approaches**

Weaknesses

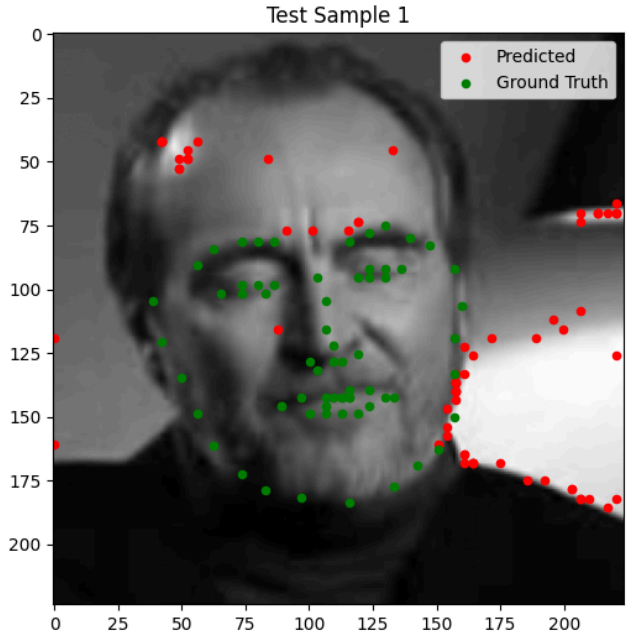
- **Hardest to implement correctly** (heatmap generation, normalization, decoding, etc.)
- Very tricky to implement the training due to the scarcity of useful information in the heatmap (most pixels are 0). Requires implementation details to avoid overfitting + training for long epochs to
- **Requires careful choices** (Gaussian size, loss weighting, resolution, decoder depth)
- **Higher memory + compute cost** due to large output tensors
- **Needs more epochs to converge (30 in my case)**

Challenges Faced :

1) US war with my home country Iran :)
1) U-net overfitting :

U-net loss dropping very fast and staying constant. Made me think deeply about the underlying signal and why learning heatmaps could be a tricky task

The part that I spent a lot of time on was implementing the U-net. Initially with naive implementation, I achieved very low loss but the predicted images did not look good. Specifically it was obvious that the model was overfitting to the training data and the predictions were clustered at edges of the image : (see below)

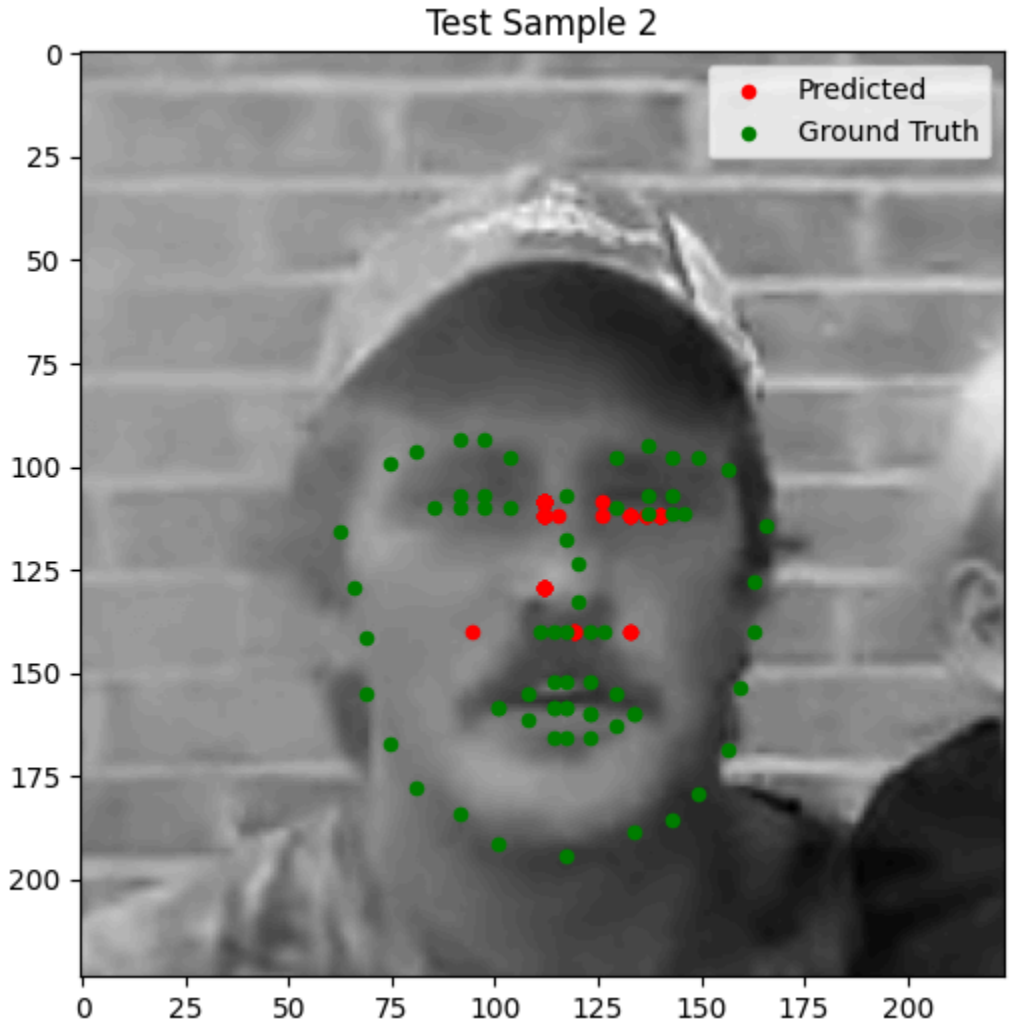


I added dropout between encoder/decoder and also added a weight-decay to the optimizer but this didn't fix the issue.

Next I added augmentation import, lr scheduler and separated train/test transforms for heatmap dataset but this didn't fix the issue.

I then realized that given MSE loss gives the mean of loss for all pixels and in a 64 by 64 heatmap with sigma=4 only 50 pixels have meaningful value, the loss is not very meaningful in training the U-net and I judged the results visually by predictions on the images.

Next I changed the loss from a simple MSE to focal heatmap.
I got this by changing objective to focal heatmap :



Next I realized MSE loss was too coarse. I changed the loss to BCE but got weird vertical strips as predictions. With only 0.5% positive pixels and pos_weight=10, the model had two stable modes: suppress everything (low confidence) or activate broad bands to catch every positive pixel. To fix this reward hacking I increased pos_weight from 10 to 100 to compensate for extreme class imbalance (now predicting near 0 everywhere gives very high loss forcing the model to learn the heatmap patterns)